

tf.tensor_scatter_nd_update Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/tensor_scatter_nd_update

Scatter updates into an existing tensor according to indices.

Function Signatures

```
tf.tensor_scatter_nd_update( tensor, indices, updates,  
name=None )
```

Code Examples

```
tf.tensor_scatter_nd_update(  
tensor, indices, updates, name=None  
)
```

```
tf.tensor_scatter_nd_update(  
tensor, indices, updates, name=None  
)
```

```
# Not implemented: tensors cannot be updated inplace.  
tensor[indices] = updates
```

```
# Not implemented: tensors cannot be updated inplace.  
tensor[indices] = updates
```

```
assert tf.rank(indices) >= 2  
index_depth = indices.shape[-1]  
batch_shape = indices.shape[:-1]  
assert index_depth <= tf.rank(tensor)  
outer_shape = tensor.shape[:index_depth]  
inner_shape = tensor.shape[index_depth:]  
assert updates.shape == batch_shape + inner_shape
```

```
assert tf.rank(indices) >= 2  
index_depth = indices.shape[-1]  
batch_shape = indices.shape[:-1]  
assert index_depth <= tf.rank(tensor)  
outer_shape = tensor.shape[:index_depth]  
inner_shape = tensor.shape[index_depth:]  
assert updates.shape == batch_shape + inner_shape
```

```
num_updates, index_depth = indices.shape.as_list()  
assert updates.shape == [num_updates]  
assert index_depth == tf.rank(tensor)`
```

```
num_updates, index_depth = indices.shape.as_list()  
assert updates.shape == [num_updates]  
assert index_depth == tf.rank(tensor)`
```

Documentation

Scatter updates into an existing tensor according to indices.

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.tensor_scatter_nd_update`, `tf.compat.v1.tensor_scatter_update`

This operation creates a new tensor by applying sparse updates to the input tensor. This is similar to an index assignment.

If an out of bound index is found on CPU, an error is returned.

> Warning: There are some GPU specific semantics for this operation.

If an out of bound index is found, the index is ignored.

The order in which updates are applied is nondeterministic, so the output will be nondeterministic if indices contains duplicates.

- If an out of bound index is found, the index is ignored.
- The order in which updates are applied is nondeterministic, so the output

will be nondeterministic if indices contains duplicates.

This operation is very similar to `tf.scatter_nd`, except that the updates are scattered onto an existing tensor (as opposed to a zero-tensor). If the memory for the existing tensor cannot be re-used, a copy is made and updated.

In general:

- indices is an integer tensor - the indices to update in tensor.
- indices has at least two axes, the last axis is the depth of the

index vectors.

- For each index vector in indices there is a corresponding entry in

updates.

- If the length of the index vectors matches the rank of the tensor, then

the index vectors each point to scalars in tensor and each update is a scalar.

- If the length of the index vectors is less than the rank of tensor, then

the index vectors each point to the slices of tensor and shape of the updates must match that slice.

Overall this leads to the following shape constraints:

Typical usage is often much simpler than this general form, and it can be better understood starting with simple examples:

Scalar updates

The simplest usage inserts scalar elements into a tensor by index.

In this case, the `index_depth` must equal the rank of the input tensor, slice each column of indices is an index into an axis of the input tensor.

In this simplest case the shape constraints are:

For example, to insert 4 scattered elements in a rank-1 tensor with 8 elements.

This scatter operation would look like this:

The length (first axis) of updates must equal the length of the indices: `num_updates`. This is the number of updates being inserted. Each scalar update is inserted into tensor at the indexed location.

For a higher rank input tensor scalar updates can be inserted by using an `index_depth` that matches `tf.rank(tensor)`:

Slice updates

When the input tensor has more than one axis scatter can be used to update entire slices.

In this case it's helpful to think of the input tensor as being a two level array-of-arrays. The shape of this two level array is split into the `outer_shape` and the `inner_shape`.

`indices` indexes into the outer level of the input tensor (`outer_shape`). and replaces the sub-array at that location with the corresponding item from the updates list. The shape of each update is `inner_shape`.

When updating a list of slices the shape constraints are:

For example, to update rows of a (6, 3) tensor:

Use an index depth of one.

The `outer_shape` is 6, the inner shape is 3:

2 rows are being indexed so 2 updates must be supplied.

Each update must be shaped to match the `inner_shape`.

Altogether this gives:

`## More slice update examples`

A tensor representing a batch of uniformly sized video clips naturally has 5 axes: `[batch_size, time, width, height, channels]`.

For example:

To replace a selection of video clips:

- Use an `index_depth` of 1 (indexing the `outer_shape`: `[batch_size]`)
- Provide updates each with a shape matching the `inner_shape`:

`[time, width, height, channels]`.

To replace the first two clips with ones:

To replace a selection of frames in the videos:

- indices must have an `index_depth` of 2 for the `outer_shape`:

`[batch_size, time]`.

- updates must be shaped like a list of images. Each update must have a

shape, matching the inner_shape: [width, height, channels].

To replace the first frame of the first three video clips:

Folded indices

In simple cases it's convenient to think of indices and updates as lists, but this is not a strict requirement. Instead of a flat num_updates, the indices and updates can be folded into a batch_shape. This batch_shape is all axes of the indices, except for the innermost index_depth axis.

updates must have a matching batch_shape (the axes before inner_shape).

With this generalization the full shape constraints are:

For example, to draw an X on a (5,5) matrix start with these indices:

Here the indices do not have a shape of [num_updates, index_depth], but a shape of batch_shape+[index_depth].

Since the index_depth is equal to the rank of tensor:

- outer_shape is (5,5)
- inner_shape is () - each update is scalar
- updates.shape is batch_shape + inner_shape == (5,2) + ()

Putting this together gives:

Returns

